手写识别

一开始

1、运行程序，在测试集上该模型的损失和精度分别是多少？
2、该模型精度很低，请分析原因。

```
313/313 [==============================] - 1s 2ms/step - loss: 0.0892 - accuracy: 0.1687
Test loss:  0.08916167169809341
Test accuracy:  0.16869999468326569
```

损失（loss）：约 0.0892 精度（accuracy）：约 16.87%
模型精度低的原因

1. **损失函数不匹配**：分类任务使用 **均方误差（MSE）**，而 MSE 更适合回归任务。分类任务需衡量“概率分布差异”，交叉熵（如 categorical_crossentropy）更高效。
2. **优化器效率低**：**SGD** 学习率固定（默认 0.01），收敛慢且易陷入局部最优，导致模型未充分训练。
3. **激活函数缺陷**：隐藏层用 **sigmoid**，其导数在输入绝对值较大时趋近于 0，引发**梯度消失**，深层网络参数更新受阻。
4. **训练不充分**：即使训练 20 轮，因上述问题，模型仍未收敛，精度提升缓慢。

第一次修改

3、修改优化算法，即将程序中以下代码行中的optimizer = 'sgd'改为optimizer = 'rmsprop'

```
model.compile(loss = 'mean_squared_error', optimizer = 'sgd', metrics = ['accuracy'])
```

```
313/313 [==============================] - 1s 3ms/step - loss: 0.0030 - accuracy: 0.9809
Test loss:  0.0030215715523809195
Test accuracy:  0.98089998960495
```

结果
损失（loss）：约 0.0030 精度（accuracy）：约 98.09%

第二次修改

5、修改损失函数，即将程序中以下代码行中的loss = 'mean_squared_error'改为loss = 'categorical_crossentropy'

```
model.compile(loss = 'mean_squared_error', optimizer = 'sgd', metrics = ['accuracy'])
```

结果

```
313/313 [==============================] - 1s 2ms/step - loss: 0.3921 - accuracy: 0.8911
Test loss:  0.39213141798973083
Test accuracy:  0.8910999894142151
```

损失（loss）：约 0.3921 精度（accuracy）：约 89.11%
测试不同损失函数
测试 categorical_crossentropy（标签 One-Hot）
也就是把这个

```
model.compile(loss='mean_squared_error', optimizer='rmsprop', metrics=['accuracy'])
```

换成

```
model.compile(loss='categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])
```

结果

```
313/313 [==============================] - 1s 2ms/step - loss: 0.3973 - accuracy: 0.8905
Test loss:  0.39728692173957825
Test accuracy:  0.890500009059906
```

损失（loss）：约 0.3972 精度（accuracy）：约 89.05%

测试 sparse_categorical_crossentropy（标签整数）

也就是把这个

```
model.compile(loss='mean_squared_error', optimizer='rmsprop', metrics=['accuracy'])
```

换成

```
model.compile(loss='sparse_categorical_crossentropy', optimizer='sgd', metrics=['accuracy'])
```

结果

```
313/313 [==============================] - 1s 3ms/step - loss: 0.3940 - accuracy: 0.8917
Test loss:  0.39397484064102173
Test accuracy:  0.891700029373169
```

损失（loss）：约 0.3940 精度（accuracy）：约 89.17%

7、尝试修改其它参数或网络结构，说明具体方法。

优化方向：

1. **激活函数**：将隐藏层 sigmoid 改为 relu（缓解梯度消失，加速训练）。
2. **正则化**：加入 Dropout（防止过拟合）。
3. **优化器**：替换为 adam（自适应学习率，收敛更快）。
4. **训练参数**：增加 epochs（如 50 轮，确保模型收敛）

具体方法

新增了 Dropout 层的导入，用于正则化处理

将隐藏层的 sigmoid 激活函数替换为 relu

在每个隐藏层后添加了 Dropout(0.2)层

随机丢弃 20% 的神经元，减少过拟合风险

将 mean_squared_error(MSE) 替换为 categorical_crossentropy(交叉熵)

将 sgd(随机梯度下降) 替换为 adam(自适应矩估计)

将训练轮数 epochs 从 20 增加到 50

结果

```
313/313 [==============================] - 1s 1ms/step - loss: 0.0996 - accuracy: 0.9831
Test loss:  0.09964914619922638
Test accuracy:  0.9830999970436096
```

损失（loss）：约 0.0996 精度（accuracy）：约 98.31%

图像识别

存在过拟合。

训练集表现：最终训练准确率高达 98.61%

测试集表现：测试准确率仅 66.96%，远低于训练集

```
Epoch 20/20
50000/50000 [==============================] - 13s 263us/step - loss: 0.0411 - acc: 0.9861
```

```
[13]: <keras.callbacks.History at 0x2b3766a2608>
```

```
[14]: score = model.evaluate(X_test, Y_test)
      print('Test loss: ', score[0])
      print('Test accuracy: ', score[1])
```

```
10000/10000 [==============================] - 2s 223us/step
Test loss:  2.54371015958786
Test acc      0.6696
```

2、尝试改进该程序，提高精度，说明具体方法及结果。